

Address wording issues related to ISO 10646

Document Number: P1139R2

Date: 2019-02-18

Audience: SG16, CWG

Author: R. Martinho Fernandes

Reply-to: cpp@rmf.io

Changelog

- revision 2:
 - fix wording issues
 - rebase on working draft
- revision 1:
 - fix rendering issues
- revision 0:
 - initial revision

Motivation

Review of some editorial fixes following the recent update of the normative reference to ISO 10646 has unearthed a series of wording issues around the subject. This paper intends to fix those issues by rewording relevant paragraphs.

Proposal

This paper addresses all of the following issues:

1. The current wording in [lex.charset] does not specify what the behaviour is for a universal-character-name without a corresponding short identifier in ISO 10646.

For example, `\U99004141` and `\U00110000`. Neither of these designates a code point in ISO 10646, but the standard is silent about this, which makes the behaviour undefined by omission.

This paper addresses this by making such uses ill-formed, maintaining consistency with the current treatment of surrogate values (`\U0000D800` is already ill-formed).

2. The current wording in [lex.charset] uses "hexadecimal value", which is confusing because a value is just a number, and hexadecimal is just a way to represent numbers; "value" alone should suffice.

This paper addresses this by removing the need for this term.

3. There is some interest in using the U+ notation (as in U+0041 or U+1F34A) to refer to Unicode code points across the entire standard.

This paper changes all the relevant wording to use U+ notation.

4. The current text includes explanations of terms from ISO 10646 (like "surrogate code point" or "control character") in normative text, which is undesirable.

This paper moves such explanations to non-normative text, and clarifies some existing explanations.

Technical Specifications

In this description, text that should be deleted is marked red and striked out; text that should be added is marked green and underlined. Apply these changes on top of the current draft, N4800 (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4800.pdf>).

Edit 5.3 [lex.charset], paragraph 2 as follows.

² The *universal-character-name* construct provides a way to name other characters.

hex-quad:

hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

universal-character-name:

\u hex-quad

\U hex-quad hex-quad

The character designated by the *universal-character-name* ~~\UNNNNNNNN~~\U00NNNNNN is that character ~~whose character short name in ISO/IEC 10646 is NNNNNNNN~~that has U+NNNNNN as a code point short identifier; the character designated by the *universal-character-name* ~~\UNNNN~~ is that character ~~whose character short name in ISO/IEC 10646 is 0000NNNN~~that has U+NNNN as a code point short identifier. ~~If the hexadecimal value for a universal-character-name corresponds to a surrogate code point (in the range 0xD800-0xDFFF, inclusive)~~If a universal-character-name does not correspond to a code point in ISO/IEC 10646 or if a universal-character-name corresponds to a surrogate code point, the program is ill-formed. Additionally, ~~if the hexadecimal value for a universal-character-name outside the c-char-sequence, s-char-sequence, or r-char-sequence of a character or string literal corresponds to a control character (in either of the ranges 0x00-0x1F or 0x7F-0x9F, both inclusive) or to a character in the basic source character set, the program is ill-formed.~~[Note: ISO/IEC 10646 code points are within the range 0x0-0x10FFFF (inclusive). A surrogate code point is a value in the range 0xD800-0xDFFF (inclusive). A control character is a character whose code point is in either of the ranges 0x0-0x1F or 0x7F-0x9F (both inclusive).—end note]

Edit 5.13.3 [lex.ccon], paragraph 3 as follows.

³ A character literal that begins with `u8`, such as `u8 'w'`, is a character literal of type `char`, known as a *UTF-8 character literal*. The value of a UTF-8 character literal is equal to its ISO 10646 code point value, provided that the code point value ~~is representable with a single UTF-8 code unit (that is, provided it is in the C0 Controls and Basic Latin Unicode block)~~can be encoded as a single UTF-8 code unit [*Note: that is, provided it is in the range 0x0-0x7F (inclusive)—end note*]. If the value is not representable with a single UTF-8 code unit, the program is ill-formed. A UTF-8 character literal containing multiple *c-chars* is ill-formed.

Edit 5.13.3 [lex.ccon], paragraph 4 as follows.

⁴ A character literal that begins with the letter `u`, such as `u 'x'`, is a character literal of type `char16_t`. The value of a `char16_t` character literal containing a single *c-char* is equal to its ISO 10646 code point value, provided that the code point value is representable with a single 16-bit code unit (~~[Note: that is, provided it is in the basic multi-lingual plane]~~the range 0x0-0xFFFF (inclusive)—end note). If the value is not representable with a single 16-bit code unit, the program is ill-formed. A `char16_t` character literal containing multiple *c-chars* is ill-formed.

Edit 5.13.3 [lex.string], paragraph 10 as follows.

¹⁰ A *string-literal* that begins with `u`, such as `u"asdf"`, is a `char16_t` string literal. A `char16_t` string literal has type “array of n `const char16_t`”, where n is the size of the string as defined below; it is initialized with the given characters. A single *c-char* may produce more than one `char16_t` character in the form of surrogate pairs. [*Note: A surrogate pair is a representation for a single code point as a sequence of two 16-bit code units.—end note*]

Edit 19.8 [cpp.predefined], item (2.4) as follows.

(2.4) `__STDC_ISO_10646__`

An integer literal of the form `yyyymmL` (for example, `199712L`). If this symbol is defined, then every character in the Unicode required set, when stored in an object of type `wchar_t`, has the same value as the ~~short identifier~~code point of that character. The Unicode required set consists of all the characters that are defined by ISO/IEC 10646, along with all amendments and technical corrigenda as of the specified year and month.